

Class 02 | 29 July

We will start sharp at 09.05 PM

C# Basics

Mastering the C# language — fundamentals features.

Today's agenda

1. Introduction to C#
2. Variables & Data Types
3. Operators
4. Control Flow
5. Loops
6. Methods
7. Arrays
8. Exception handling

01

Introduction to C#

What is C#, and how a C# program is structured.

What is C#?

- C# is a modern, object-oriented programming language developed by Microsoft for building a wide variety of applications.
- We can build modern web applications, desktop applications, Games, Mobile applications.
- C# is easy to learn, object-oriented, Strongly Typed, Cross-platform.

.NET vs C#

- **C# is programming language but .NET is a platform that understands that language and runs your application**
- **C# used to write code but .NET is used to compile & execute.**



Top-level statements vs. the classic **Main**

Traditional

```
using System;
namespace HelloApp
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello");
        }
    }
}
```

Top-level statements

```
// Program.cs — the whole file
Console.WriteLine("Hello");
```

Same program. The compiler generates the `Main` for you. This is the modern default.

Create your first Hello world! program in Visual Studio

- Open visual studio
- Select Create new project
- Select Console application
- give your project name and solution name.
- select .net version
- and create

02

Variables & Data Types

What is Variable?

- Variables are containers for storing data values.
- Think of it as a labeled box where you can store information.
- A variable stores data that can change during program execution.
- Variable declaration:
 - `<data type> <variable name> = <value>;`
 - **F.ex** : `int age = 25;`

Common Data Types in C#

- **int** -> **Whole Numbers** -> **200**
- **double** -> **Decimal numbers** -> **10.15**
- **string** -> **Text** -> “[ASP.NET](#) Course”
- **char** -> **Single character** -> ‘C’
- **bool** -> **True/False** -> **true**

Notes: Strings are always enclosed in double quotes (")

Variables Naming Rules

- **Use camelCase (First letter lowercase, subsequent words capitalized)**
 - `string courseName = “ASP.NET”`
- **_camelCase (Leading underscore followed by camelCase)**
 - `string _courseName = “ASP.NET”`
- **PascalCase (First letter capitalized, all subsequent words capitalized)**
 - `string CourseName = “ASP.NET”`

Type Inference

- **Type Inference is a feature where the C# compiler automatically determines the data type of a variable based on the value assigned to it.**
- **`var age = 25;`**
- **`var name = "Mehraj";`**

The compiler treats these exactly as:

- **`int age = 25;`**
- **`string name = "Mehraj";`**

Constants

- **A constant stores a value that cannot be changed.**
 - `const double Pi = 3.14159;`
 - `Pi = 5;`
- **Compile-time Error**

03

Operators

What is Operator?

- **An operator is a symbol that tells the computer to perform an operation on one or more values.**
- **Examples of operations include:**
 - **Addition**
 - **Subtraction**
 - **Comparison**
 - **Logical decisions**

Four families of operators

Arithmetic

+ - * / %

Comparison

== != < > <= >=

Logical

&& || !

Assignment

= += -= *= /=

Homeworks 1

- Takes two numbers from the user.

Prints:

- Sum
- Difference
- Product
- Division
- Remainder

Homeworks 2

- **Create a program that asks the user for their age and prints:**
 - **Is the user an adult (a user is adult if age is greater or equal than 18)?**
 - **Is the user exactly 18 years old?**
 - **Is the user younger than 18?**

Example Output

- **Adult: True**
- **Exactly 18: False**
- **Under 18: False**

04

Control Flow

What is Conditional Statements?

- **A conditional statement allows a program to make decisions based on a condition.**



The if Statement

Syntax

if (condition)

{

// Code to execute if the condition is true

}

The if Statement

Example

```
int age = 20;  
  
if (age >= 18)  
{  
    Console.WriteLine("You are an adult.");  
}
```

The if...else Statement

```
int age = 15;

if (age >= 18)
{
    Console.WriteLine("You can vote.");
}

else
{
    Console.WriteLine("You cannot vote yet.");
}
```


Classic **switch** vs. **switch expression**

Classic statement

```
switch (status)
{
    case 1:
        Console.WriteLine("Not yet");
        break;
    default:
        Console.WriteLine("Posted");
        break;
}
```

Switch expression · C# 8+

```
string label = status switch
{
    1    => "Not yet",
    2 => "Live",
    _    => "Unknown"
};
```

Home works 3, 4, 5, 6

- **Exercise 1: Even or Odd**
- **Exercise 3: Positive, Negative, or Zero**
- **Exercise 5: Largest of Two Numbers**
- **Exercise 6: Grade Calculator**



05

Loop

What is Loop?

- A loop allows us to execute the same block of code multiple times without writing it repeatedly.
- Let's print 1 to 10.



The ‘for’ Loop

Syntax

for (initialization; condition; update)

{

```
// Code to repeat
```

}

The ‘while’ Loop

Syntax

while (condition)

{

```
// Code
```

}

The ‘do while’ Loop

Syntax

do

{

```
//
```

}

while (condition);

06

Methods

What is Method?

- A method is a block of code that performs a specific task.
- Instead of writing the same code multiple times, we write it once inside a method and call it whenever we need it.
- Write once, use many times.



Syntax

```
returnType MethodName(parameters)
{
    // Code
}
```

- returnType is basically a data type

Homework 7, 8

- Create a Method to return the sum of two numbers.
- Create a Method to Print 1 to 10.



07

Arrays

What is Array?

- An array is a collection of elements of the same data type stored together.
- Instead of creating multiple variables:
- `int mark1 = 85;`
- `int mark2 = 90;`
- `int mark3 = 78;`

Syntax

```
dataType[] arrayName = new dataType[size];
```

Example:

```
int[] numbers = new int[5];
```

This creates an array that can store 5 integers.

Syntax

Initializing an Array

You can assign values when creating the array.

```
int[] numbers = { 10, 20, 30, 40, 50 };
```



08

Exception Handling

What is Exception?

- **An exception is an error that occurs while a program is running.**
- **Without exception handling, the application may stop unexpectedly.**

Examples:

- **Dividing by zero**
- **File not found**
- **Accessing an invalid array index**

Exception handling

```
try
{
    if (tweet.Length > 280)
        throw new Exception("Too long");
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message); // handle it
}
finally
{
    Console.WriteLine("Done"); // always runs
}
```

OPEN FLOOR

Questions?

WHAT'S NEXT

The .NET ecosystem (Class 04) and OOP (Class 06)

The two things we set aside today. Now you have the language they're built on.